

TALENTBRIDGE

PRODUCT REQUIREMENT DOCUMENT (PRD)

UMHackathon 2026

<https://github.com/gankaiken/UM-Hackathon>

1. Introduction:

TalentBridge AI tackles the core problems in today's hiring systems. Instead of relying on basic keyword matching in resumes, it uses a multi-agent AI pipeline to create a blind evaluation layer—assessing candidates based on practical skills, reasoning, and initiative rather than just what's written on paper.

1.1. Purpose:

The system is built around an asynchronous, event-driven agent architecture that aims to **reduce human bias** in recruitment, **identify integrity issues** such as cheating, and **offer meaningful development opportunities** to all applicants. Its core purpose is to prioritize proven ability and potential over formal qualifications.

1. Architecture:

- Modular Next.js App Router with an internal "Agent Bus."

2. Data Flows:

- Unstructured Job Description (JD)
- Structured Dimensions
- Adaptive Interview
- Behavioral Sentinel
- Bias-Blind Verdict.

3. Model Process:

- [Z.AI](#) (GLM-5.1) serves as the core reasoning engine for 7 AI agents.

4. Role of Reference:

- The Job Description (JD) acts as the source-of-truth for the entire session lifecycle.

1.2. Background:

Today's hiring still leans heavily on resumes—static profiles that can be easily optimized or even generated by AI, yet don't reflect a candidate's real abilities. The result is poor hiring decisions, higher turnover, and overlooked talent, especially in fast-growing markets like Malaysia.

1. Previous Version:

- a. Current hiring practices **rely heavily on resumes** and **keyword-based filtering** systems, which often fail to reflect a candidate's true abilities. This approach makes it easy to game the system while overlooking individuals with real potential but weaker presentation. As a result, companies face **biased selection**, **inconsistent evaluation**, and higher turnover, while capable talent—especially in emerging markets—remains underrecognized.

2. Changes in Major Architectural Components

- a. Every candidate is able to take the AI interview which will analyze them based on a range of aspects coded in our model prompt.
- b. HR directly gets a summary of every candidate including their strengths and weaknesses and 'verdict cards' which give HR a quick overview and comparison of every candidate.

3. New capabilities that are only introduced in this

- a. **AI interview** which analyzes every strength and weakness from different aspect (refer to 7 AI models) with cheating detector implemented
- b. **Verdict cards** provided which greatly increase the efficiency of hiring process
- c. **Job Description Mapper** which automatically defines 5 dimensions for every candidate to be analyzed

1.3. Target Stakeholder

Stakeholders	Roles	Expectations
HR Managers	Primary User	High-fidelity "Verdict Cards," 60% reduction in screen time.
Founders/ CXOs	Stakeholder	Data-backed hiring decisions, strong employer branding.
Candidates	End User	A fair, conversational chance to prove talent; zero-ghosting feedback.
Dev Team	Provider	Modular agent code, strict schema

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview:

Type	Details
System	Web Application (AI-powered hiring platform)
Architecture	Architecture: Monolithic Web Application with AI Processing Pipeline

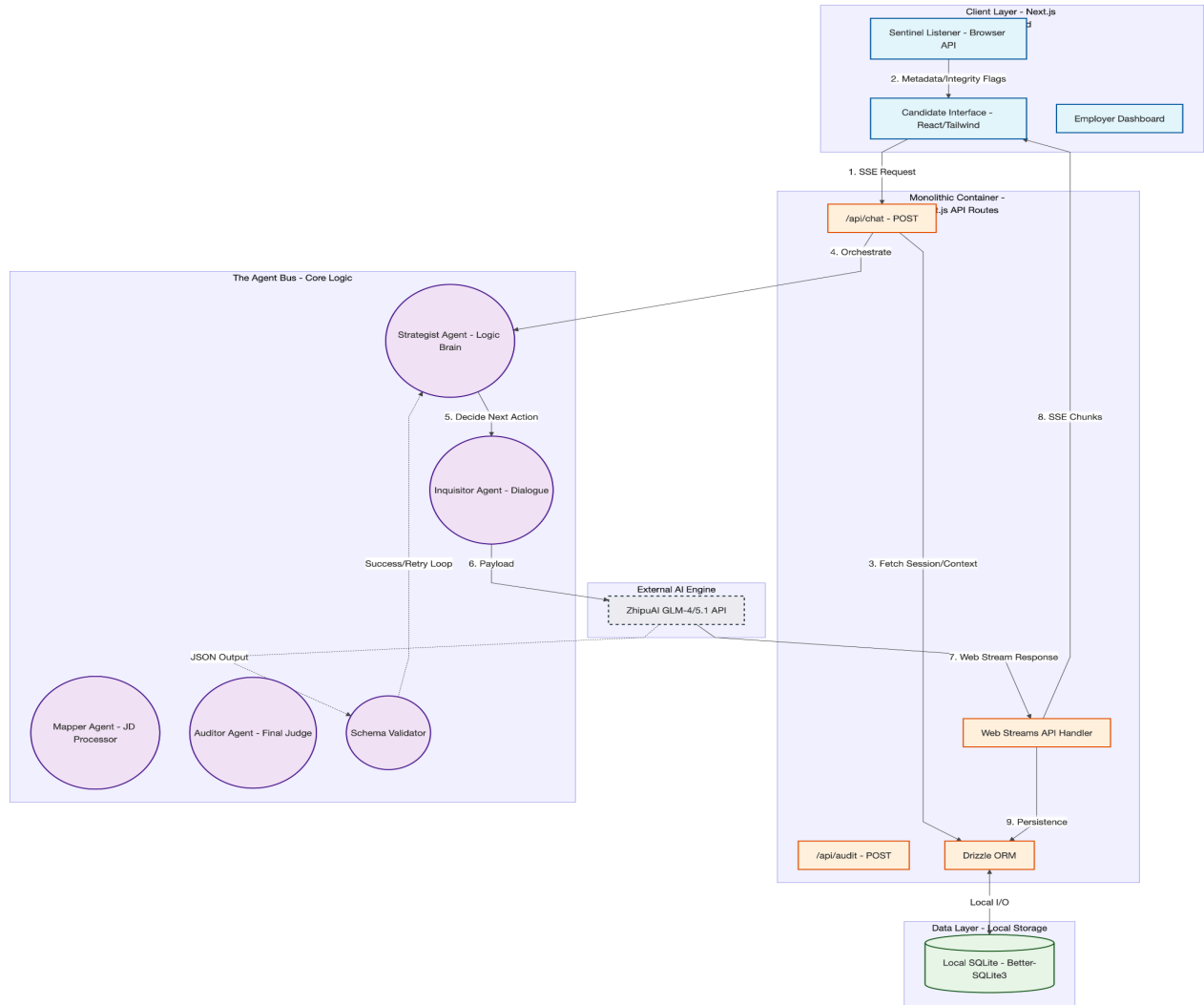
2.1.2 LLM as Service Layer

TalentBridge treats the GLM API not as a simple "text completer," but as a State Machine.

We use the API to:

- Reason: Deciding which skill to probe next based on the "Coverage Map."
- Audit: Cross-referencing Sentinel flags with linguistic style shifts.

2.1.3 Dependency Diagram



1. Prompt Construction & Context Window

- **Dynamic Construction:** Prompts are generated dynamically via templates. The 5 Core Dimensions extracted by the Mapper (`mapper.core_dimensions`) are hard-coded into the Strategist Agent's System Prompt to strictly bind its reasoning boundaries.
- **Metadata Injection:** Browser events caught by the Sentinel are processed into an "Integrity State" (e.g., `stage_2_alert`). This is injected directly as a "CURRENT STATE" prefix into the latest user message to trigger a "Reality Check."
- **Full Context Retention:** The context window receives the full, unsummarized conversation history in a [Turn X] Role: Message format. Summarization is intentionally avoided to preserve verbatim "Key Evidence" for the final Auditor.

2. Response Parsing & Execution Pipeline

- **Stream Handling:** The system receives responses from the Zhipu API via native Web Streams. A custom backend `transformStream` uses `TextDecoder` to parse the chunks and repackages them into Server-Sent Events (SSE) for the frontend UI.
- **JSON Validation & Retry:** When structured data is required, the system enforces Zhipu's native JSON Mode (`response_format: { type: 'json_object' }`) combined with a Regex fallback to clean Markdown.
- **Self-Correction:** If the Schema Validator detects missing or corrupted JSON keys, an `executeAgent` wrapper automatically triggers an Exponential Backoff Retry Loop (maximum 2 retries) before failing.

3. Token Limitation & Chunking Strategy

- **Turn-Based Meltdown:** Instead of text chunking (which destroys audit evidence), the system enforces a strict 20-turn limit. Once reached, the Strategist overrides all logic to output a `close_session` command, preventing context window overflow.
- **Role-Specific Max Tokens:** Token limits are strictly capped based on the Agent's Scope of Work (SOW): Strategist is capped at 1200, Inquisitor at 256 (for snappy, human-like replies), and Auditor at 5000 (for full transcript review).

4. Major API Calls & System Interaction Flow

- **Core API Call:** The frontend Candidate UI makes a single, primary API call: `POST /api/chat` to the Next.js backend, listening for an SSE response.
- **End-to-End Interaction Flow:**
 1. Frontend sends candidate text and Sentinel events to `/api/chat`.
 2. Backend uses Drizzle ORM to fetch session history from the database.
 3. The Strategist Agent analyzes the history to determine the next probing angle.
 4. The Inquisitor Agent makes a `POST` call to the external Zhipu GLM API.
 5. The Next.js backend receives the LLM response, streams it to the Frontend via SSE, and simultaneously persists the new transcript row into the database via Drizzle ORM.

5. Distributed Components & Deployment Strategy

- **Monolithic Container Architecture:** The system is not distributed across multiple microservices. It is deployed as a single Monolithic Docker Container hosted on Google Cloud Run.
- **Ultra-Low Latency Persistence:** To achieve real-time AI performance without network bottlenecks, the database is not hosted on an external RDS. The system uses a local file-based SQLite (Better-SQLite3) database bundled inside the container, allowing for <1ms synchronous reads/writes.

2.2.2. Sequence Diagram

Step	Actor	Action	System Response
1	HR Manager	Pastes raw Job Description text into the Employer Dashboard.	Frontend sends JD text to /api/map-jd.
2	Mapper Agent	/api/map-jd calls GLM-5.1 with JD Mapper prompt.	Returns MapperResult JSON: 5 weighted dimensions (e.g., Problem-Solving 30%, Communication 20%).
3	Dimension QA Agent	Validates the 5 dimensions for coverage and non-redundancy.	Approves or requests a re-map. Result stored in jd_cache with a unique Role Key (UUID).
4	System	Unique interview link generated and shared with candidate.	New session row created in sessions table with role_key, candidate metadata, and empty coverage map.
5	Candidate	Opens link, enters name, begins interview.	Inquisitor Agent sends a warm opening question based on the first uncovered dimension.
6	Candidate	Sends a chat message response.	Frontend POSTs to /api/interview with the full conversation history + coverage map + current Sentinel state.
7	Strategist Agent	Analyzes coverage map; identifies next skill gap.	Outputs a strategist_json block (stored in transcripts for audit) directing the Inquisitor's next probe.

8	Inquisitor Agent	Formulates next question using Strategist's reasoning.	Streams tokens back to candidate UI via SSE. Coverage map updated in sessions table.
9	Sentinel (Browser)	Continuously monitors tab switches, copy-pastes, focus loss.	POSTs SentinelData events to /api/sentinel; appended to session's sentinel_blob.
10	Reality Check	If Sentinel detects suspicious activity, Inquisitor is notified.	System injects a meta-prompt asking the candidate to verbally confirm a key point (integrity check).
11	Candidate/System	Session ends (all dimensions covered or candidate submits).	Full transcript + sentinel_blob sent to /api/verdict.
12	Auditor Agent	Strips grammar/slang bias; scores on Action Nodes only.	Returns VerdictResult JSON with scores, Key Evidence quotes, triage color, and upskill paths.
13	Schema Validator	Validates VerdictResult. If malformed, triggers retry.	On success, verdict written to sessions table. Retry loop fires up to 3 times on failure.
14	HR Manager	Views Verdict Card on HR Dashboard.	Visual PDF-style card rendered: GREEN/AMBER/RED triage, dimension scores, evidence quotes, upskill recommendations.

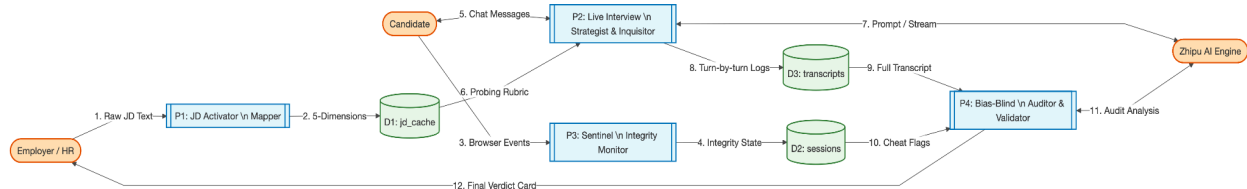
2.2. Technological Stack

- 2.2.1. Frontend: Next.js 15 (App Router) with Tailwind CSS v4. Animations via Motion (Framer Motion).
- 2.2.2. Backend: Serverless Node.js functions within Next.js. Streaming handled via Web Streams API.

- 2.2.3. Database: SQLite (Better-SQLite3) managed by Drizzle ORM. Synchronous, file-based for ultra-low latency in local deployments.
- 2.2.4. AI Engine: ZhipuAI (GLM-4) for multi-lingual (BM/EN/Manglish) support and JSON mode.
- 2.2.5. Cloud & Infra: Containerized via Docker, deployed on Google Cloud Run via the AI Studio platform.

2.3. Key Data Flows

2.3.1. Here, do show the how data are moving through the system and how its structure and stored



2.3.2. Data Flow Diagram (DFD)

- Illustrate the data movement across the entire system that includes external entities (users), processes, external service and databases
- Show how proper laws are met (e.g. Black Hole)

Entity/Process	Input	Output	Law Compliance
External: Recruiter	Raw JD Text	Trigger: Extract Rubric	No Black Hole: High-level text is utilized immediately by the Mapper.
P1: The Mapper	Raw JD Text	MapperResult (JSON)	Consistency: Input matches the domain logic of the Job.
External: Candidate	Resume + Chat Msg	Session Context	No Miracle: AI responses are strictly gated by Candidate input.

P2: The Sentinel	Window Events	SentinelData	Integrity: Acts as a "write-only" listener to prevent tampering.
P3: Live Interview (Strategist & Inquisitor)	Chat Msg + Dimensions + Integrity State	Next Action Prompt & Streaming Chat	No Grey Hole: Strategist ensures sufficient data coverage before passing to Auditor.
D3: Transcripts (Data Store)	Streaming Chat Logs	Full Transcript	Persistence: Synchronous local DB writes prevent data loss.
P4: The Auditor & Validator	Full Transcript	Verdict Card	Closure / No Miracle: Every score generated MUST be backed by a Key Evidence quote from the transcript.

2.3.3. Normalized Database Schema

- jd_cache: Stores extracted dimensions and weights. Unique ID serves as the "Role Key."
- sessions: Tracks candidate metadata, live coverage maps, the "Sentinel Data" blob, and final verdicts.
- transcripts: Log of every turn. crucially stores strategist_json for auditing the AI's own decision logic.

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects. That enables them to focus on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product:

#	Features	Description
---	----------	-------------

1	JD Activator	Convert any text JD into an active interview portal.
2	Live Inquisitor	Multi-turn streaming chat that mirrors the candidate's language style.
3	Cheat-Guard (Sentinel)	Real-time monitoring of tab switches and copy-pastes
4	Verdict Generator	Visual PDF-style cards with triage (GREEN/AMBER/RED) and upskill paths

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	Requirements (e.g.)	Implementation (e.g)
Persistence	Interview sessions can be resumed if browser closes unexpectedly or the connection drops mid-session	Sessions can be resumed if the browser closes
Integrity	The system must ensure all behavioral tracking data and interview transcripts remain secure and unalterable to accurately detect cheating and provide a trustworthy evaluation.	Tamper-evident Sentinel logs (locked once session ends)
Latency	The system must provide a highly responsive, real-time conversational experience,	Character-streaming ensures < 500ms time-to-first-token for conversation

	ensuring minimal delay between the candidate's input and the AI's initial response.	
Scalability	The system should handle concurrent interview sessions without shared-state conflicts.	Each session is fully isolated in its own SQLite row. Cloud Run auto-scales container instances horizontally. For v2, migration to PostgreSQL (via PlanetScale or Neon) is planned to support multi-instance deployments.
Reliability	The platform must remain functional for demos even without the Z.AI API key.	A mock fallback layer (mockMappers, mockInquisitor) returns pre-seeded structured data for all 7 agents. The UI is indistinguishable from a live session, ensuring demo reliability.
Maintainability	Agent prompts must be version-controlled and independently updatable without full redeployment.	All agent system prompts are stored as TypeScript constants in /lib/agents/. Changes require only a file edit and a Cloud Run redeploy (< 2 minutes via CI/CD).
Token Latency	Prompt construction must not exceed 80% of the model's context window to leave headroom for the response.	Token budget is enforced: JD block (≤ 800 tokens) + rolling window ($\leq 2,000$ tokens) + Sentinel state (≤ 200 tokens) = $\leq 3,000$ input tokens, leaving $\geq 1,024$ tokens for the response.
Cost Efficiency	API costs must remain predictable and bounded per session.	Each session is capped at a maximum of 15 Inquisitor turns. The Auditor is called once per session (not per turn).

		Estimated cost per completed session: < USD 0.05 at current GLM-5.1 pricing.
--	--	---

3.3. Out of Scope / Future Enhancements

- a. The Grand Vision:
- b. Voice Forensics: Using pitch and pause analysis to detect nervousness or external coaching.
- c. Company Culture "Digital Twin": Uploading team Slack/Discord logs to the Mapper to tune the "Cultural Fit" dimension.
- d. Integrations: Auto-posting Green-tier candidates directly to Workday or Greenhouse.

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation:

KPI	Target	Measurement Method
Time-to-first-token (Inquisitor)	< 500ms	Browser Performance API (window.performance.now()) measured from POST submission to first streamed character.
Session completion rate	> 80% of started sessions	SQLite query: sessions WHERE status = 'completed' / sessions WHERE status != 'abandoned'.
Verdict Card generation success rate	> 95%	Schema Validator pass rate logged per session; retry loop invocations tracked.
Sentinel false-positive rate	< 10%	Manual review of 20 flagged sessions during hackathon testing; cross-referenced with audio/screen observation.

JD Mapper dimension quality	> 4/5 HR approval rating	HR evaluators rate the 5 auto-generated dimensions for relevance and coverage on a Likert scale.
-----------------------------	--------------------------	--

4.2. Priority Matrix:

Feature	Impact	Effort	Priority
JD Activator (Mapper + QA)	High — unlocks all downstream agents	Medium	P0 — Must Have
Live Inquisitor (Streaming Chat)	High — core candidate experience	High	P0 — Must Have
Verdict Generator (Auditor + Schema Validator)	High — primary HR deliverable	Medium	P0 — Must Have
Cheat-Guard (Sentinel)	High — differentiating integrity feature	Low	P1 — Should Have
Mock Fallback Layer	Medium — demo reliability	Low	P1 — Should Have
Session Resumability	Medium — production reliability	Low	P2 — Nice to Have
PDF Export of Verdict Card	Low — convenience feature	Low	P2 — Nice to Have

4.3. Monitoring:

- Application Logs: All API route handlers log request duration, agent name, token count, and success/failure status to stdout (captured by Cloud Run's built-in logging).
- Error Tracking: Schema Validator failures and GLM API errors are caught, logged with full context (session ID, prompt excerpt), and trigger the mock fallback gracefully.

- Health Endpoint: A /api/health route checks SQLite connectivity and Z.AI API reachability, returning a JSON status object for uptime monitoring.
- Cost Monitoring: GLM API token usage is logged per session and aggregated daily. Alerts are set at 80% of the monthly budget threshold.

4.4. Assumptions:

- 4.4.1. Requires ZHIPU_API_KEY for full pipeline; falls back to mockMappers for offline/demo reliability.
- 4.4.2. Assumes modern evergreen browsers (Chrome/Safari/Edge) to support visibilitychange API for Sentinel.

4.5. External Dependencies: Do list all the external tools that are applied to your system to function. [e.g. below]

Tools	Purpose	Risks
Z Ai GLM API	It should be the core LLM reasoning engine - that will handle the prompt inference, multi-step reasoning and generating structured output	High- in case of GLM API rate limitation - core product will fail. So, implement a mitigation strategy [e.g. graceful degradation state from users]
Google Maps API	Tracking rider location and optimize the route	Medium level- if real time tracking is being disabled.

5. Project Management & Team Contributions

5.1. Project Timeline:

Phase	Days	Key Deliverables
Planning & Architecture	Day 1–2	Finalize PRD and System Analysis Doc. Define agent schemas and JSON contracts. Set up Next.js monorepo, Docker environment, Drizzle ORM schema, and Cloud Run CI/CD pipeline.
Core Agent Development	Day 3–4	Implement JD Mapper, Dimension QA, and jd_cache persistence. Build /api/map-jd and /api/interview routes. Integrate Z.AI GLM-5.1 with streaming.
Interview UX & Sentinel	Day 5–6	Build the candidate chat UI with Framer Motion streaming animations. Implement Sentinel browser layer (visibilitychange, paste, focus events). Build /api/sentinel route.
Auditor & Verdict Card	Day 7–8	Implement Auditor agent, Schema Validator, and retry loop. Build the Verdict Card UI (GREEN/AMBER/RED triage, evidence quotes, upskill paths). Integrate /api/verdict.
Integration, Testing & Polish	Day 9	End-to-end integration testing across all 7 agents. Fix edge cases (malformed JSON, API timeouts). Add mock fallback layer. Performance testing for streaming latency.
Demo Preparation & Submission	Day 10	Final UI polish. Record demo video. Write submission README.

5.2. Team Members Role: Highlight the role of team members [e.g. Person A – Role Backend Lead – Fast API Microservices & Schema Design]

Person	Role	Responsibilities
Member A	Full-Stack Lead	Overall system architecture, Z.AI API integration, Docker & Cloud Run deployment, and supervising the integration of global payroll compliance modules
Member B	Frontend Lead / UX Engineer	Next.js App Router structure, Tailwind CSS v4 design system, and developing the HR Dashboard for Enterprise and Startup clients.
Member C	Backend & Database Engineer	Drizzle ORM schema design, SQLite optimization, session resumability, and implementing the Global Compliance API stubs.
Member D	AI architect	Engineering the 7 agent prompts, managing Deep-learning models for skill-matching , and implementing the grounding layer to mitigate AI hallucinations
Member E	QA & Documentation Lead	End-to-end test scenarios, managing the QATD, verifying the 80% AI Output Pass Rate, and producing the demo video and submission packaging.

5.3. Recommendations:

Area	Recommendation	Rationale
Database	Migrate from SQLite to PostgreSQL (e.g., Neon serverless) for v2.	SQLite is single-writer by design. Multi-instance Cloud Run deployments require a networked database to avoid write conflicts under concurrent load.

Caching	Introduce Redis (e.g., Upstash) to cache JD Mapper results (jd_cache).	The same job description is often processed multiple times. Caching the MapperResult eliminates redundant GLM API calls, reducing latency and cost by an estimated 30–40% per role.
AI Model	Implement a model router to fall back from GLM-5.1 to GPT-4o or Claude Sonnet on API failure.	A multi-provider fallback eliminates the single point of failure on the Z.AI API, improving the system's overall SLA to > 99.9%.
Observability	Add OpenTelemetry tracing across all API routes and agent calls.	Distributed tracing enables per-turn latency attribution, making it straightforward to identify which agent is the bottleneck in a slow session.
Security	Implement rate limiting (e.g., Upstash Ratelimit) on all /api/* routes.	Prevents abuse of the GLM API through the TalentBridge proxy, protecting both cost and availability.
Verdict Delivery	Add email delivery of Verdict Cards (e.g., via Resend or SendGrid).	Eliminates the requirement for HR to log into the dashboard; reduces time-to-decision for async hiring workflows.